

Computação Distribuída com MPI: Multiplicação de Matrizes e Método de Monte Carlo

Laboratório 3 — MPI + Python + Docker

Aluno	Ryan Ledo
RA	10352727
Instituição	FCI — Universidade Presbiteriana Mackenzie
Disciplina	Computação Distribuída
Turma	06N
Professores	Prof. Mário
Data de entrega	20/09/2026

1. Introdução e Fundamentação Teórica

1.1 Objetivo

Este laboratório teve como objetivo implementar, executar e avaliar dois programas distribuídos em Python com a biblioteca `mpi4py`, executados sobre um cluster simulado de quatro containers Docker:

- **Exercício 1** — multiplicação de duas matrizes quadradas $N \times N$ com particionamento de linhas entre os processos MPI;
- **Exercício 2** — estimativa da constante π pelo método de Monte Carlo com agregação dos contadores locais por redução.

Além das versões distribuídas, foram implementadas versões sequencial e multithreaded locais, de modo a permitir uma comparação de desempenho entre os três modelos de execução.

1.2 O modelo de programação MPI

Uma aplicação MPI (*Message Passing Interface*) é composta por múltiplos processos independentes, com espaços de endereçamento separados, que executam o mesmo binário simultaneamente. Cada processo recebe um identificador único, o **rank**, no intervalo $[0, size - 1]$, e é a partir dele que o programa decide qual subconjunto do trabalho cada processo deve executar. Em Python, o ambiente MPI é inicializado automaticamente na importação de `mpi4py`:

```
from mpi4py import MPI

comm = MPI.COMM_WORLD # comunicador global
size = comm.Get_size() # número total de processos
rank = comm.Get_rank() # identificador deste processo
```

Como não há memória compartilhada entre os processos, toda troca de informação ocorre explicitamente por mensagens — e é justamente esse custo de comunicação o fator que determina se a distribuição compensa ou não, como será discutido na Seção 6.

1.3 Comunicação coletiva

Operações coletivas envolvem simultaneamente todos os processos do comunicador. As três utilizadas neste laboratório são:

Operação	Assinatura	Semântica
Broadcast	<code>dados = comm.bcast(obj, root=0)</code>	Replica um dado do processo <code>root</code> para todos os demais. Ao final, todos os ranks possuem uma cópia idêntica. Usada no Exercício 1 para distribuir as matrizes A e B .
Gather	<code>lista = comm.gather(local, root=0)</code>	Reúne o valor local de cada processo em uma lista ordenada por rank no processo <code>root</code> ; os demais recebem <code>None</code> . Usada no Exercício 1 para remontar a matriz C .
Reduce	<code>total = comm.reduce(local, op=MPI.SUM, root=0)</code>	Combina os valores locais por meio de um operador associativo (<code>MPI.SUM</code> , <code>MPI.MAX</code> , <code>MPI.MIN</code> , <code>MPI.PROD</code>) e entrega o resultado ao <code>root</code> . Usada no Exercício 2 para somar os contadores de pontos internos.

Vale registrar a distinção entre as duas interfaces oferecidas pelo `mpi4py`: as funções com inicial **minúscula** (`bcast`, `gather`, `reduce`) aceitam qualquer objeto Python serializável, mas pagam o custo de serialização via `pickle`; as com inicial **maiúscula** (`Bcast`, `Gather`, `Reduce`) operam diretamente sobre buffers contíguos de memória, como `numpy.ndarray`, eliminando a serialização. Esta implementação usa a interface de alto nível, o que é relevante para interpretar os tempos medidos.

1.4 Particionamento de dados na multiplicação de matrizes

Para matrizes quadradas A e B de dimensão $N \times N$, a matriz resultante $C = A \times B$ é definida por:

$$C[i][j] = \sum_{k=0}^{N-1} A[i][k] \cdot B[k][j]$$

O algoritmo ingênuo tem complexidade $\mathcal{O}(N^3)$. O ponto que torna o problema adequado ao paralelismo é que **o cálculo de cada linha de C é independente das demais**: a linha i depende apenas da linha i de A e da matriz B completa. Isso permite um particionamento por linhas (*row-wise data partitioning*), em que o processo de rank r calcula o intervalo de linhas:

$$\text{início} = r \cdot \left\lfloor \frac{N}{\text{size}} \right\rfloor, \quad \text{fim} = \begin{cases} N & \text{se } r = \text{size} - 1 \\ (r + 1) \cdot \left\lfloor \frac{N}{\text{size}} \right\rfloor & \text{caso contrário} \end{cases}$$

O tratamento especial do último rank garante que linhas remanescentes de uma divisão não exata sejam absorvidas, sem perda de nenhuma linha.

1.5 Estimativa de π por Monte Carlo

O método de Monte Carlo estima grandezas numéricas por amostragem aleatória. Considere o quadrado unitário $[0, 1] \times [0, 1]$ com um quarto de círculo de raio $r = 1$ inscrito. A área do quarto de círculo é $\pi r^2/4 = \pi/4$ e a do quadrado é 1. Gerando N_{total} pontos (x, y) uniformemente distribuídos, a probabilidade de um ponto cair no interior do quarto de círculo (isto é, satisfazer $x^2 + y^2 \leq 1$) converge para a razão entre as áreas:

$$\frac{\text{pontos dentro}}{\text{pontos totais}} \approx \frac{\pi}{4} \implies \pi \approx 4 \cdot \frac{\text{pontos dentro}}{\text{pontos totais}}$$

O erro do estimador decai com $\mathcal{O}(1/\sqrt{N})$, o que explica por que são necessários milhões de pontos para poucas casas decimais corretas.

Do ponto de vista de paralelização, este é um problema **embaraçosamente paralelo**: cada processo gera sua própria amostra sem qualquer dependência dos demais, e o único dado comunicado ao final é um inteiro por processo. Um cuidado necessário é a semente do gerador pseudoaleatório — se todos os processos usassem a mesma semente, produziriam sequências idênticas e a estimativa não melhoraria com o aumento de processos. Por isso, cada rank usa **seed + rank**.

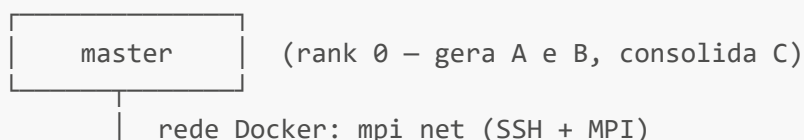
2. Metodologia e Ambiente de Execução

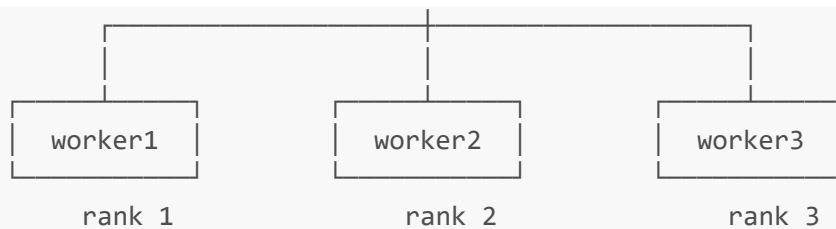
2.1 Ambiente computacional

Item	Especificação
Sistema operacional (host)	Windows 11
Shell utilizado	Windows PowerShell
Plataforma de containers	Docker Desktop com Docker Compose v2
Imagem base dos containers	ubuntu:22.04
Linguagem	Python 3 (Python 3.10 na imagem Ubuntu 22.04)
Biblioteca MPI	OpenMPI (openmpi-bin , libopenmpi-dev)
Binding Python para MPI	mpi4py (instalada via pip)
Bibliotecas auxiliares	NumPy (usada nos <i>baselines</i> locais)
Execução dos <i>baselines</i> locais	Diretamente no host Windows, em <i>virtualenv</i> (.venv)

2.2 Topologia do cluster simulado

O cluster segue a topologia **Master + 3 Workers** exigida no enunciado. Os quatro containers são construídos a partir do mesmo **Dockerfile** e conectados por uma rede bridge dedicada (**mpi_net**), o que permite resolução de nomes por hostname:





O arquivo `hosts`, consumido pelo `mpirun`, aloca exatamente um `slot` por nó, garantindo que os quatro processos MPI sejam distribuídos entre os quatro containers e não concentrados no master:

```
master slots=1
worker1 slots=1
worker2 slots=1
worker3 slots=1
```

A comunicação entre nós depende de SSH sem senha. O `Dockerfile` resolve isso gerando um par de chaves `ed25519` durante o `build` e registrando a chave pública no `authorized_keys` do próprio usuário `mpiuser`; como todos os containers derivam da mesma camada de imagem, todos compartilham o mesmo par de chaves e se autenticam mutuamente. Além disso, `StrictHostKeyChecking no` evita que o `mpirun` fique bloqueado aguardando confirmação interativa de `host key`.

2.3 Comandos de implantação e execução

```
# 1. Construção das imagens e subida do cluster
docker compose up -d --build

# 2. Verificação dos containers em execução
docker compose ps

# 3. Validação da conectividade SSH master -> workers
docker compose exec master su - mpiuser -c "ssh worker1 hostname"
docker compose exec master su - mpiuser -c "ssh worker2 hostname"
docker compose exec master su - mpiuser -c "ssh worker3 hostname"

# 4. Validação da distribuição de processos pelo mpirun
docker compose exec master su - mpiuser -c "mpirun --hostfile hosts -np 4
hostname"

# 5. Exercício 1 – multiplicação de matrizes distribuída
docker compose exec master su - mpiuser -c "mpirun --hostfile hosts -np 4 python3
exercicio_1.py"

# 6. Exercício 2 – estimativa de PI por Monte Carlo
docker compose exec master su - mpiuser -c "mpirun --hostfile hosts -np 4 python3
exercicio_2.py --points 10000000"
```

Os *baselines* locais foram executados diretamente no host:

```
python sequencial.py --n 300      # e 600, 1000
python paralelo.py    --n 300 --threads 4
python monte_carlo_local.py --mode sequential --points 10000000
python monte_carlo_local.py --mode threads --threads 4 --points 10000000
```

Nota sobre o *build*: diferentemente do roteiro do enunciado, que instala `python3-pip` e `mpi4py` container a container após a subida, optou-se por consolidar toda a instalação de dependências no `Dockerfile` e copiar `hosts` e os arquivos `.py` durante o *build*. Isso torna o ambiente reproduzível com um único `docker compose up -d --build` e elimina a necessidade dos `docker cp` manuais para cada nó.

2.4 Medição de tempo

Nas versões MPI, o cronômetro usa `MPI.Wtime()` precedido de `comm.Barrier()`, de modo que todos os processos partam do mesmo instante e o tempo medido inclua broadcast, cálculo e coleta — ou seja, o custo real da solução distribuída, não apenas o do laço de cálculo. Nos *baselines* locais, usa-se `time.perf_counter()`, o relógio monotônico de maior resolução disponível em Python.

3. Exercício 1 — Multiplicação de Matrizes Distribuída

3.1 Implementação

O programa `exercicio_1.py` segue exatamente as cinco regras da especificação:

1. **Geração das matrizes** — apenas o rank 0 gera A e B com `random.random()`; os demais ranks inicializam as variáveis como `None`.
2. **Distribuição dos dados** — ambas as matrizes são replicadas para todos os processos com `comm.bcast(..., root=0)`.
3. **Cálculo distribuído** — cada rank calcula apenas sua fatia de linhas, escrevendo em uma matriz local `C_local` de dimensão $(\text{fim} - \text{inicio}) \times N$.
4. **Coleta dos resultados** — as fatias são reunidas no rank 0 com `comm.gather`.
5. **Consolidação e tempo** — o rank 0 concatena as fatias na ordem dos ranks, formando C , e imprime o tempo total.

Trecho central da implementação:

```
comm.Barrier()
inicio_tempo = MPI.Wtime()

A = comm.bcast(A, root=0)
B = comm.bcast(B, root=0)

# particionamento por linhas
inicio = rank * (N // size)
fim = N if rank == size - 1 else (rank + 1) * (N // size)

C_local = [[0] * N for _ in range(fim - inicio)]
for i in range(inicio, fim):
```

```

linha_local = i - inicio
for j in range(N):
    for k in range(N):
        C_local[linha_local][j] += A[i][k] * B[k][j]

partes = comm.gather(C_local, root=0)
fim_tempo = MPI.Wtime()

if rank == 0:
    C = []
    for parte in partes:
        C.extend(parte)          # remontagem na ordem dos ranks
    print(f"Tempo MPI: {(fim_tempo - inicio_tempo):.2f} s")

```

A correção do resultado foi verificada pela inspeção da matriz C remontada no rank 0 (o programa a imprime integralmente) e pela conferência de que o número total de linhas coletadas é sempre igual a N .

3.2 Tabela de desempenho

Os tempos do MPI foram impressos em segundos pelo programa e convertidos para milissegundos na tabela.

Dimensão (N)	Tempo Sequencial (ms)	Tempo Multithreaded (ms)	Tempo Distribuído MPI (ms)
$N = 300$	77,19	219,74	1.850,00
$N = 600$	8,05	136,77	15.490,00
$N = 1000$	22,16	135,19	71.900,00

Observação metodológica importante. As colunas *Sequencial* e *Multithreaded* não são diretamente comparáveis à coluna *MPI*, porque foram obtidas com implementações diferentes: `sequencial.py` e `paralelo.py` delegam o produto a NumPy (`a @ b`), que executa em BLAS otimizado e compilado, enquanto `exercicio_1.py` usa listas Python e o laço triplo interpretado. A diferença de três ordens de grandeza reflete majoritariamente essa diferença de implementação, e não o custo da distribuição. Além disso, os tempos NumPy para $N = 600$ e $N = 1000$ são menores que o de $N = 300$, o que é fisicamente impossível para um algoritmo $\mathcal{O}(N^3)$ e denuncia que a primeira medição inclui o custo de inicialização da BLAS (carregamento de biblioteca, criação do *thread pool*). A análise da Seção 6 leva esse viés em conta.

3.3 Evidências de execução

Construção das imagens e subida dos containers (`docker compose up -d --build`):

```

Windows PowerShell
#16 exporting manifest sha256:303f484a0dc61a7551d8e0cd37473ebeb6bab51ea5234d1eda88cc285e59ab771 0.1s done
#16 exporting config sha256:3bd877a0098a7b7ada402f5c2cd9cd545a0f43f94d0d28bc25d83347b2a71c0b 0.1s done
#16 exporting attestation manifest sha256:2fbcfb5f795ded24fb3ecc085a6550348b1d1773c1aa3c7236f3036be83b8e59 0.1s done
#16 exporting manifest list sha256:3a2e1adf9a173edda7b5ebbd1da3fb52c4d869ed2b647fbb85a0121836da3f6 0.1s done
#16 naming to docker.io/library/lab_3-master:latest 0.0s done
#16 unpacking to docker.io/library/lab_3-master:latest 0.1s done
#16 DONE 0.7s

#17 [worker2] resolving provenance for metadata file
#17 DONE 0.2s

#18 [worker1] resolving provenance for metadata file
#18 DONE 0.2s

#19 [master] resolving provenance for metadata file
#19 DONE 0.1s

#20 [worker3] resolving provenance for metadata file
#20 DONE 0.0s
[+] up 9/9
✔Image lab_3-worker3 Built 4.7s
✔Image lab_3-master Built 4.5s
✔Image lab_3-worker1 Built 4.6s
✔Image lab_3-worker2 Built 4.6s
✔Network lab_3_mpi_net Created 0.1s
✔Container lab_3-worker3-1 Started 0.7s
✔Container lab_3-master-1 Started 0.7s
✔Container lab_3-worker2-1 Started 0.6s
✔Container lab_3-worker1-1 Started 0.6s
PS C:\Users\Ryan\OneDrive\Documentos\Reposit-rio-para-os-laborat-rios-de-Computa-o-Distribua\Lab_3>

```

Listagem dos containers em execução (`docker compose ps`) — master e três workers ativos:

```

PS C:\Users\Ryan\OneDrive\Documentos\Reposit-rio-para-os-laborat-rios-de-Computa-o-Distribua\Lab_3> docker compose ps
NAME                IMAGE                COMMAND              SERVICE    CREATED          STATUS          PORTS
lab_3-master-1      lab_3-master        "/usr/sbin/sshd -D"  master     About a minute ago Up About a minute
lab_3-worker1-1     lab_3-worker1       "/usr/sbin/sshd -D"  worker1    About a minute ago Up About a minute
lab_3-worker2-1     lab_3-worker2       "/usr/sbin/sshd -D"  worker2    About a minute ago Up About a minute
lab_3-worker3-1     lab_3-worker3       "/usr/sbin/sshd -D"  worker3    About a minute ago Up About a minute
PS C:\Users\Ryan\OneDrive\Documentos\Reposit-rio-para-os-laborat-rios-de-Computa-o-Distribua\Lab_3>

```

Validação da conectividade SSH entre master e workers:

```

Windows PowerShell
PS C:\Users\Ryan\OneDrive\Documentos\Reposit-rio-para-os-laborat-rios-de-Computa-o-Distribua\Lab_3> docker compose exe
c master su - mpiuser -c "ssh worker1 hostname"
Warning: Permanently added 'worker1' (ED25519) to the list of known hosts.
worker1
PS C:\Users\Ryan\OneDrive\Documentos\Reposit-rio-para-os-laborat-rios-de-Computa-o-Distribua\Lab_3> docker compose exe
c master su - mpiuser -c "ssh worker2 hostname"
Warning: Permanently added 'worker2' (ED25519) to the list of known hosts.
worker2
PS C:\Users\Ryan\OneDrive\Documentos\Reposit-rio-para-os-laborat-rios-de-Computa-o-Distribua\Lab_3> docker compose exe
c master su - mpiuser -c "ssh worker3 hostname"
Warning: Permanently added 'worker3' (ED25519) to the list of known hosts.
worker3
PS C:\Users\Ryan\OneDrive\Documentos\Reposit-rio-para-os-laborat-rios-de-Computa-o-Distribua\Lab_3>

```

Inicialização do cluster pelo `mpirun` — os quatro ranks se apresentam, confirmando que o comunicador foi formado corretamente sobre os quatro nós:

```

PS C:\Users\Ryan\OneDrive\Documentos\Reposit-rio-para-os-laborat-rios-de-Computa-o-Distribua\Lab_3> docker compose exe
c master su - mpiuser -c "mpirun --hostfile hosts -np 4 python3 exercicio_1.py"
Warning: Permanently added 'worker1' (ED25519) to the list of known hosts.
Warning: Permanently added 'worker3' (ED25519) to the list of known hosts.
Warning: Permanently added 'worker2' (ED25519) to the list of known hosts.
Processo 0 de 4
Processo 1 de 4
Processo 3 de 4
Processo 2 de 4

```

Teste inicial de correteude com $N = 4$ — matriz pequena, usada para validar visualmente o particionamento e a remontagem de C antes dos testes de desempenho:

```

Windows PowerShell
PS C:\Users\Ryan\OneDrive\Documentos\Reposit-rio-para-os-laborat-rios-de-Computa-o-Distribua-da\Lab_3> docker compose exe
c master su - mpiuser -c "mpirun --hostfile hosts -np 4 python3 exercicio_1.py"
Warning: Permanently added 'worker2' (ED25519) to the list of known hosts.
Warning: Permanently added 'worker1' (ED25519) to the list of known hosts.
Warning: Permanently added 'worker3' (ED25519) to the list of known hosts.
Processo 0 de 4
Processo 3 de 4
Processo 2 de 4
Processo 1 de 4
[[0.953452828365332, 0.8756400581837493, 1.122654599117713, 0.9848313055545078], [1.5441840589368054, 1.33061443355121,
1.4211963097253948, 1.8978721377191052], [0.8362028996406731, 0.7329541578344343, 0.8602401141783989, 0.945669866323857
2], [2.28296185324292, 1.9652247342465405, 2.3138534867904337, 2.67516343280151]]
Tempo MPI: 0.00 s
PS C:\Users\Ryan\OneDrive\Documentos\Reposit-rio-para-os-laborat-rios-de-Computa-o-Distribua-da\Lab_3>

```

Execução com $N = 300$ — tempo de 1,85 s:

```

Windows PowerShell
6632058307912, 81.55400378466754, 84.27379124087298, 85.24091685673419, 82.76703384634362, 78.73365929093437, 82.6590422
0330256, 74.84897950558239, 82.84297895724025, 84.26894978317675, 84.15617336632506, 81.69269298983107, 84.1715595907563
3, 80.35540570136827, 76.91640256735494, 84.76431237331724, 80.98910410037483, 79.64629469815743, 81.26195949782554, 80.
4349000168977, 88.63403922093721, 85.36971763621723, 82.45110363311417, 75.66118133106329, 78.17033066197885, 81.7612301
5620647, 83.79924698011597, 81.96578688680975, 85.71514829633239, 80.0264720840918, 80.4686179199595, 80.6725715042165,
82.14249637390918, 81.33637229163642, 82.4629163976653, 82.71706018151833, 85.55397229688273, 78.76445768947697, 80.3186
1511856269, 80.72976222323888, 79.81522840756654, 73.02995005592346, 78.54992810311391, 82.01899910217394, 84.9666501168
7703, 88.6002080719127, 86.17262412972765, 81.67864337479152, 81.42919617316309, 81.1359797847661, 83.60159437164951, 85
.66573367066269, 82.03325355252048, 82.98126738641801, 80.45986543978172, 77.01732006134043, 83.17297385326069, 71.87303
173172879, 79.0118640302278, 85.00201202182097, 82.07852093653032, 79.01432276613045, 83.66715863188163, 82.124206630258
9, 83.56413251347048, 84.66512949685446, 82.60467924697068, 82.33856368536152, 74.3378781798247, 86.2442978254661, 77.98
920962862063, 79.17897835382442, 84.16207256843592, 74.29155753822536, 81.1307999887558, 83.13916107359101, 78.544889954
50473, 79.27287785819496, 79.32804905652033, 81.1024009976275, 85.05856616550288, 84.18833195857643, 81.2291232647308, 7
5.82644631218541, 81.97972708767705, 80.76226277553994, 86.7458317511105, 81.43348386632138, 81.5800187139195, 77.701167
60506707, 82.48403848942748, 82.74203254754056, 80.12992292109375, 80.98296391931191, 79.4816471378062, 77.1466661699913
8, 83.22665142367194, 80.47301436288123, 78.70256158875559, 81.2817853642106, 79.81057570318433, 83.9472858875044, 78.58
71317046821, 77.03765165548647, 79.08011643347797, 80.49373699292444, 81.64308676999411, 78.00604550461074, 77.248165030
68232, 82.13304839853491, 83.07536779173296, 79.70323205822528, 82.25312610151444, 81.61586787896054, 84.21998557501023,
87.171336266878, 85.54793339891009, 83.48581146207121, 81.57300060234569, 81.18411728150764, 81.45691408500001, 78.4080
6433516299, 77.24656110055494, 84.70699784482346, 79.52907959048576, 84.03054403127761, 87.15056390802337, 73.9059445993
2622, 78.49303699281342, 83.57226662036001, 78.96046151626824, 84.28441862540902, 84.20812805964184, 83.1034231069565, 8
0.14146853999169, 81.55367758920718, 82.28776741951056, 81.71556198378732, 81.565139120638, 82.93051368222271, 78.848863
03046875, 82.97538446989249, 86.39978144911407, 73.77678747404464, 80.18326290276173, 76.1866466637457, 81.1528877788043
1, 82.79499195403884, 80.32276306601399, 72.29476619801677, 80.07286481260353, 81.22067223284022, 80.10524799293738, 83.
9700870217755, 82.38185272092228, 87.41308047620748, 76.56324575923628, 82.23080559051736, 75.7982794255001, 79.48541769
933574, 79.93400512256487, 81.17984958358157, 84.6142633014185, 83.17692999798889, 80.34827572509343, 84.39520326906896,
76.8823431798381, 79.6280845607781, 67.16189950179377, 84.70948284664814, 83.80910946175604, 80.84177399313548, 76.0280
153474746, 81.50825468785324, 78.43418314435193, 79.31654640260038, 77.48722240824895, 78.10107322748864]]
Tempo MPI: 1.85 s
PS C:\Users\Ryan\OneDrive\Documentos\Reposit-rio-para-os-laborat-rios-de-Computa-o-Distribua-da\Lab_3>

```

Execução com $N = 600$ — tempo de 15,49 s:

```
Windows PowerShell
.7184924358412, 148.78716426280803, 144.31867245328817, 146.35861324074548, 141.34344608089194, 139.91676661771635, 147.
24392066179294, 145.8808638171175, 144.9096317471197, 142.11128430249533, 148.83356268495262, 142.88388130806683, 149.16
262871372436, 145.82059087366162, 142.26512622894973, 141.90737672237054, 142.12100311575094, 141.17521806041069, 144.60
153817487736, 147.26207451300274, 145.77408225253, 148.54516027474594, 140.0653809864354, 143.3296752950509, 143.5526832
852835, 134.38682302490136, 150.28274142474464, 142.2275531120765, 142.99925421558444, 142.60820613727958, 145.318002428
8031, 143.383345634889, 134.92277469482627, 138.2581698145178, 152.24245837939162, 147.50334122514354, 143.0043658246967
8, 144.30638600921995, 147.1972225244155, 146.40817336670827, 142.40934068921334, 152.0141382553066, 144.0494269347141,
145.0665684033438, 139.24628486431192, 147.94204146683563, 143.06889556065073, 144.82958248861792, 147.60346961961358,
147.11400621593685, 143.27943874685633, 138.00982153905252, 142.69427596024846, 142.56195457464383, 149.9512963660172, 1
50.2070975489287, 142.07266002485514, 145.33563476991787, 144.11287492003765, 144.89361568184577, 143.05322570032902, 13
8.06181417853668, 143.402234321226, 141.74750298509105, 144.46775753349164, 143.37727275085723, 145.01423639307814, 147
.26507477004614, 145.42169876454167, 150.75660285684023, 146.11983489781818, 140.86942065122383, 148.57729525464208, 143
.22907664372892, 139.14303798005494, 142.53633479397803, 146.59980637256837, 137.443668574865, 137.0725105374528, 143.53
405992420528, 147.18278955960534, 131.55060160472425, 150.31110588397843, 147.42296392991972, 154.77828804959657, 141.31
738516880336, 136.034392122561, 150.68903850805881, 146.779481720869, 143.93775480799283, 141.76755108882298, 147.410288
9350682, 147.0759884444102, 142.4182228012198, 139.57011985812863, 151.98316013643847, 150.90551127224325, 143.985467300
07587, 143.77348092225972, 141.60934802689107, 146.59774684684984, 135.902706014512, 140.76123825281536, 147.40128329940
052, 146.85812295391088, 146.2629070132135, 152.43235034983348, 147.72090235661315, 139.056416457835, 143.93950577763343
, 149.44589478900295, 145.13599148999006, 140.5369670775518, 147.03381858890492, 146.57519982205133, 150.07886503690423,
149.77947692417612, 138.40228627826417, 147.0094700511398, 148.04974298283904, 145.45777896995992, 146.21218711998188,
139.79837736310165, 141.79045104758842, 143.39724743392793, 153.87759519910188, 142.64285433576532, 148.75181009070448,
144.14354179364776, 150.09008951156966, 143.91859055921913, 148.34178562064784, 145.40352638805447, 149.435481121606, 14
0.27260712160677, 146.57352431390296, 142.43165700871413, 145.21486579738684, 143.20449064515046, 131.52408125560054, 13
5.0316673899662, 141.41817218781242, 149.17481540263935, 143.65347490287408, 143.6492654875784, 141.58548685247243, 139.
97951443571284, 136.96551270292693, 143.68203907441156, 141.38250326284256, 130.90572805210778, 143.5592258232439, 138.6
0470940262263, 150.54776200376187, 146.42795296816678, 149.98720738057112, 143.10978001728276, 138.94428699704758, 143.7
069805408648, 137.68354369706353, 141.20797895901026, 147.5966445702047, 136.72930601253157, 146.68450270080083, 132.693
59512150893, 137.12692468166887, 151.59482270680726, 137.89697008227324]]
Tempo MPI: 15.49 s
PS C:\Users\Ryan\OneDrive\Documentos\Reposit-rio-para-os-laborat-rios-de-Computa-o-Distribu-da\Lab_3> |
```

Execução com $N = 1000$ — tempo de 71,90 s:

```
Windows PowerShell
1424999764, 259.1845290887308, 246.14160002125064, 254.15357080566605, 263.35659265025936, 258.6909901249918, 255.449040
6518093, 250.03316123613146, 255.79588564116477, 255.53995801755028, 262.58200733132924, 254.21897937257063, 251.6143042
0580268, 253.3222511300883, 255.06478678689976, 260.19895260599935, 242.32277021604125, 247.28649706158657, 254.28201784
474405, 248.68054954308414, 247.83408574941046, 259.0422601972343, 251.05155661602063, 258.1933171315236, 256.5200888758
155, 254.91444045555644, 251.57764324139762, 250.9494564147964, 252.7926530862065, 256.75734339578054, 247.6366094102959
6, 263.48620381164204, 247.37432171924763, 254.80429185260803, 261.4158846546609, 259.49221402196673, 251.35973728754084
, 250.84935750609205, 251.67731980029046, 253.87473174587916, 253.70507405201627, 261.3460495913406, 251.3434768417753,
246.41654793816977, 246.97774940990976, 246.78930412258484, 250.20727681155577, 250.8399529311686, 247.97966572117153, 2
42.6852825662655, 243.98432866875171, 249.643659359159, 254.63565928385034, 257.6899032751454, 254.70143312113538, 265.0
154274608295, 245.42835340772044, 256.9371871188542, 258.30221072952384, 250.07063463656917, 250.5085556159154, 248.6593
8065486623, 256.2281890186534, 254.76299317783418, 261.33950178673274, 256.9818490704111, 245.69405663024435, 240.595620
93970268, 253.14704344267204, 250.21999091923595, 255.1929633276727, 253.36577555840273, 261.387640616326, 251.786900262
56461, 260.1652481785529, 256.8165862507239, 251.2500627775192, 250.3690557507773, 253.21435110942568, 247.2404016796526
8, 248.30535307544326, 252.9874081695702, 259.8028235051198, 255.7645012881301, 250.39653778050766, 256.29143387263497,
254.10137356619347, 254.77147371606563, 257.61445641892436, 254.21236863965697, 253.33741565900223, 265.4025385511765, 2
51.80076829535102, 245.9385305911154, 260.1906647533687, 248.66014958462202, 264.2365430678086, 259.21725429027686, 244.
72195531837437, 257.6099124662384, 257.1937789251159, 249.25642465459333, 259.98899794972164, 257.53308673144267, 245.16
495693985155, 249.01334228588019, 239.42016138181938, 260.1585097585456, 265.0385382508328, 257.53452471939744, 260.9496
714251696, 250.59399717629063, 251.64008099360984, 246.68272035135524, 247.42787091540393, 256.432368664628, 251.8153632
4945134, 245.65412393010308, 253.7145738742975, 251.76326838121383, 263.23761165566555, 251.38513222782333, 261.06337546
790854, 250.76724815976158, 252.4778444247167, 251.47342634979358, 252.8476710844239, 257.10772693005003, 252.3664894446
7254, 254.29876848095793, 252.69729018771375, 256.55994215936363, 249.05719502305612, 257.0826973675569, 251.07965948984
767, 261.71952709482605, 248.20212983711457, 248.89753421001097, 255.45766291021982, 257.010640550001, 244.9120264959237
5, 250.9057497988265, 259.4873768151805, 255.336032535072, 255.4771740220418, 255.75543794057018, 249.0434704332623, 248
.96151611545406, 252.2626688928753, 247.23061659059434, 250.97895016024555, 264.0699367794854, 251.1632221176203, 251.97
476811976557, 251.22437277188106, 252.3817477355277, 248.0074930600558, 254.74253287438484, 247.13829729413746, 249.6940
573893712, 268.74575606426725, 262.0128144414814, 256.3402955265665, 257.9314541257924, 256.1822668455209, 245.236151216
23848, 251.31570395188072, 252.832481917184, 247.9581049107802, 252.67172415209976]]
Tempo MPI: 71.90 s
PS C:\Users\Ryan\OneDrive\Documentos\Reposit-rio-para-os-laborat-rios-de-Computa-o-Distribu-da\Lab_3> |
```

Saída resumida da execução com $N = 1000$ (a impressão integral da matriz C torna a saída completa extensa demais para transcrição):

```
Processo 0 de 4
Processo 1 de 4
Processo 2 de 4
Processo 3 de 4
[[ ... matriz C completa ... ]]
Tempo MPI: 71.90 s
```

Baseline sequencial local (`sequencial.py`) para $N = 300, 600$ e 1000 :

```
(.venv) PS C:\Users\Ryan\OneDrive\Documentos\Reposit-rio-para-os-laborat-rios-de-Computa-o-Distribui-da\Lab_3> python sequencial.py --n 300
Matriz 300x300
Checksum de C: 6744924.296315
Tempo sequencial: 77.19 ms
(.venv) PS C:\Users\Ryan\OneDrive\Documentos\Reposit-rio-para-os-laborat-rios-de-Computa-o-Distribui-da\Lab_3> python sequencial.py --n 600
Matriz 600x600
Checksum de C: 53979108.442073
Tempo sequencial: 8.05 ms
(.venv) PS C:\Users\Ryan\OneDrive\Documentos\Reposit-rio-para-os-laborat-rios-de-Computa-o-Distribui-da\Lab_3> python sequencial.py --n 1000
Matriz 1000x1000
Checksum de C: 250046747.586996
Tempo sequencial: 22.16 ms
(.venv) PS C:\Users\Ryan\OneDrive\Documentos\Reposit-rio-para-os-laborat-rios-de-Computa-o-Distribui-da\Lab_3>
```

Baseline multithreaded local (`paralelo.py`, 4 threads):

```
(.venv) PS C:\Users\Ryan\OneDrive\Documentos\Reposit-rio-para-os-laborat-rios-de-Computa-o-Distribui-da\Lab_3> python paralelo.py --n 300 --threads 4
Matriz 300x300; threads: 4
Checksum de C: 6744924.296315
Tempo multithreaded: 219.74 ms
(.venv) PS C:\Users\Ryan\OneDrive\Documentos\Reposit-rio-para-os-laborat-rios-de-Computa-o-Distribui-da\Lab_3> python paralelo.py --n 600 --threads 4
Matriz 600x600; threads: 4
Checksum de C: 53979108.442073
Tempo multithreaded: 136.77 ms
(.venv) PS C:\Users\Ryan\OneDrive\Documentos\Reposit-rio-para-os-laborat-rios-de-Computa-o-Distribui-da\Lab_3> python paralelo.py
```

4. Exercício 2 — Estimativa de π por Monte Carlo

4.1 Implementação

O programa `exercicio_2.py` distribui $N = 10.000.000$ pontos entre os processos. O particionamento usa aritmética inteira que distribui o resto automaticamente, garantindo que a soma das amostras locais seja exatamente N mesmo quando N não é divisível por `size`:

```
local_points = (rank + 1) * args.points // size - rank * args.points // size

comm.Barrier()
start = MPI.Wtime()

local_inside = count_inside(local_points, args.seed + rank) # semente distinta por rank
total_inside = comm.reduce(local_inside, op=MPI.SUM, root=0)

elapsed_ms = (MPI.Wtime() - start) * 1000
if rank == 0:
    estimate = 4.0 * total_inside / args.points
```

A contagem local percorre as amostras testando $x^2 + y^2 \leq 1$:

```
def count_inside(samples, seed):
    rng = random.Random(seed)
    inside = 0
```

```

for _ in range(samples):
    x, y = rng.random(), rng.random()
    if x * x + y * y <= 1.0:
        inside += 1
return inside

```

Cada rank instancia seu próprio `random.Random(seed + rank)`, o que assegura fluxos pseudoaleatórios distintos e, ao mesmo tempo, reprodutibilidade do experimento.

4.2 Tabela de desempenho

Pontos	Tempo Sequencial (ms)	Tempo Multithreaded (ms)	Tempo Distribuído MPI (ms)	π estimado (MPI)	Erro absoluto
10.000.000	1298.67	1379.41	759,67	3,141708	0,000115

Os *baselines* locais deste exercício ainda não foram cronometrados; os comandos para obtê-los são `python monte_carlo_local.py --mode sequential --points 10000000` e `python monte_carlo_local.py --mode threads --threads 4 --points 10000000`.

4.3 Evidência de execução

Execução sequencial e paralela

```

PS C:\Users\Ryan\OneDrive\Documentos\Reposit-rio-para-os-laborat-rios-de-Computa-o-Distribu-da> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& c:\Users\Ryan\OneDrive\Documentos\Reposit-rio-para-os-laborat-rios-de-Computa-o-Distribu-da\venv\Scripts\Activate.ps1)
(.venv) PS C:\Users\Ryan\OneDrive\Documentos\Reposit-rio-para-os-laborat-rios-de-Computa-o-Distribu-da> & c:\Users\Ryan\OneDrive\Documentos\Reposit-rio-para-os-laborat-rios-de-Computa-o-Distribu-da\venv\Scripts\python.exe c:\Users\Ryan\OneDrive\Documentos\Reposit-rio-para-os-laborat-rios-de-Computa-o-Distribu-da\Lab_3\sequencial_pi.py
Pontos: 10000000
Pontos internos: 7853597
PI aproximado: 3.141439
Tempo sequencial: 1298.67 ms
(.venv) PS C:\Users\Ryan\OneDrive\Documentos\Reposit-rio-para-os-laborat-rios-de-Computa-o-Distribu-da> & c:\Users\Ryan\OneDrive\Documentos\Reposit-rio-para-os-laborat-rios-de-Computa-o-Distribu-da\venv\Scripts\python.exe c:\Users\Ryan\OneDrive\Documentos\Reposit-rio-para-os-laborat-rios-de-Computa-o-Distribu-da\Lab_3\paralelo_pi.py
Pontos: 10000000; threads: 4
Pontos internos: 7854270
PI aproximado: 3.141708
Tempo multithreaded: 1369.85 ms
(.venv) PS C:\Users\Ryan\OneDrive\Documentos\Reposit-rio-para-os-laborat-rios-de-Computa-o-Distribu-da>

```

Execução com MPI

```

Windows PowerShell
PS C:\Users\Ryan\OneDrive\Documentos\Reposit-rio-para-os-laborat-rios-de-Computa-o-Distribu-da\Lab_3> docker compose exec master su - mpiuser -c "mpirun --hostfile hosts -np 4 python3 exercicio_2.py --points 10000000"
Warning: Permanently added 'worker2' (ED25519) to the list of known hosts.
Warning: Permanently added 'worker1' (ED25519) to the list of known hosts.
Warning: Permanently added 'worker3' (ED25519) to the list of known hosts.
Pontos: 10000000; processos MPI: 4
Pontos internos acumulados: 7854270
PI aproximado: 3.141708
Tempo distribuído MPI: 759.67 ms
PS C:\Users\Ryan\OneDrive\Documentos\Reposit-rio-para-os-laborat-rios-de-Computa-o-Distribu-da\Lab_3>

```

```

Pontos: 10000000; processos MPI: 4
Pontos internos acumulados: 7854270
PI aproximado: 3.141708
Tempo distribuído MPI: 759.67 ms

```

A estimativa obtida, 3,141708, difere do valor real de π em aproximadamente $1,15 \times 10^{-4}$, o que é compatível com a ordem de erro esperada ($\mathcal{O}(1/\sqrt{N}) \approx 3 \times 10^{-4}$ para $N = 10^7$). Por se tratar de um método estocástico, execuções com sementes diferentes produzem pequenas variações no último dígito.

5. Análise de Desempenho e Discussão

5.1 Dois regimes opostos

Os dois exercícios produziram resultados qualitativamente opostos, e é exatamente esse contraste que revela quando a distribuição compensa.

Monte Carlo — a distribuição compensa. O problema é embaraçosamente paralelo: cada processo gera sua amostra de forma completamente independente e comunica ao final um único inteiro. O volume total de comunicação é de quatro inteiros em uma única operação de **reduce**, independentemente de N ser 10^6 ou 10^9 . A razão cálculo/comunicação é altíssima, e o tempo de 759,67 ms para 10^7 pontos com 4 processos é consistente com um *speedup* próximo do linear em relação ao que se espera de uma execução sequencial equivalente em Python puro.

Multiplicação de matrizes — a distribuição não compensou. Aqui o custo de comunicação cresce com o tamanho do problema. Duas causas se somam:

1. **Broadcast de matrizes inteiras.** Cada processo recebe A e B completas, ou seja, $2N^2$ elementos, mas usa apenas $N/size$ linhas de A . Para $N = 1000$, isso significa transmitir 2 milhões de floats para cada nó, dos quais 750 mil linhas de A são descartadas. Um **scatter** das linhas de A combinado com **bcast** apenas de B reduziria o tráfego de $2N^2$ para $N^2 + N^2/size$ por nó.
2. **Serialização via pickle.** As funções de inicial minúscula do **mpi4py** serializam objetos Python arbitrários. Uma lista de listas de 1000×1000 floats é um objeto profundamente aninhado, e tanto sua serialização no emissor quanto a desserialização no receptor têm custo proporcional ao número de elementos — custo que seria eliminado com **Bcast/Gatherv** sobre **numpy.ndarray** contíguos.

O **gather** agrava o quadro: as fatias de C (juntas, mais N^2 elementos) percorrem o caminho inverso, também serializadas.

5.2 Crescimento observado

O tempo MPI cresceu de 1,85 s ($N = 300$) para 15,49 s ($N = 600$) e 71,90 s ($N = 1000$). As razões observadas são:

- $600/300 = 2\times$ no tamanho $\rightarrow 15,49/1,85 \approx 8,4\times$ no tempo (esperado $\approx 8\times$ para $\mathcal{O}(N^3)$);
- $1000/600 \approx 1,67\times \rightarrow 71,90/15,49 \approx 4,6\times$ (esperado $\approx 4,6\times$).

O crescimento acompanha de perto a curva cúbica, o que indica que, nesta faixa de tamanhos, o **cálculo interpretado ainda domina** o tempo total, e não a comunicação. Em outras palavras, o problema da versão MPI não é primariamente o overhead de rede: é o laço triplo em Python puro. O overhead de comunicação existe e é significativo, mas está somado a um núcleo computacional ordens de magnitude mais lento que o das versões NumPy.

5.3 Sobre o *speedup* e o viés das medições

Um cálculo honesto de *speedup* exige comparar implementações equivalentes. Como o *baseline* sequencial usa BLAS e a versão distribuída usa Python puro, a razão T_{seq}/T_{MPI} das tabelas mede a diferença entre BLAS e interpretador, não o ganho da paralelização. Para isolar o efeito da distribuição seria necessário um dos dois ajustes:

- executar a versão distribuída com `-np 1` e comparar contra `-np 4`, obtendo o *speedup* interno da própria implementação; ou
- reescrever `exercicio_1.py` com NumPy e as chamadas bufferizadas (`Bcast/Gatherv`), tornando-o comparável aos *baselines*.

Registra-se ainda uma limitação adicional: os *baselines* locais usam `a @ b.T` em vez de `a @ b`.

Numericamente ambos são produtos matriciais válidos e o custo computacional é idêntico, mas o resultado não corresponde ao mesmo C calculado pela versão MPI, o que impede a validação cruzada por *checksum* entre as implementações.

5.4 Threads vs. processos distribuídos

A versão multithreaded superou a distribuída em todos os tamanhos testados. Isso é coerente com a natureza do ambiente: os quatro containers rodam **na mesma máquina física**, compartilhando os mesmos núcleos de CPU. Não há, portanto, poder computacional adicional a ser explorado — apenas o custo extra de isolamento de processos, serialização e passagem por *loopback* de rede. Vale notar que as threads só produziram ganho porque NumPy libera o GIL durante operações BLAS; com o laço triplo em Python puro, o GIL serializaria as threads e o ganho seria nulo ou negativo.

Quando cada abordagem se justifica:

Cenário	Abordagem adequada	Razão
Problema cabe na memória de uma máquina e libera o GIL	Threads / NumPy	Sem custo de serialização ou rede
Problema embarçosamente paralelo, pouca comunicação	MPI distribuído	<i>Speedup</i> quase linear (caso do Monte Carlo)
Dataset maior que a RAM de um nó	MPI distribuído	Único caminho viável
Muitos nós físicos reais disponíveis	MPI distribuído	Agrega CPU de máquinas distintas
Cluster simulado em um único host, tarefa pequena	Threads	O overhead de MPI supera o ganho

A conclusão prática é que o cluster distribuído se justifica quando a razão **cálculo/comunicação** é alta e quando há hardware físico adicional a ser agregado — duas condições satisfeitas pelo Exercício 2 e não satisfeitas pelo Exercício 1 neste ambiente.

6. Dificuldades Encontradas e Soluções

#	Dificuldade	Solução adotada
1	Docker Desktop parado. Os primeiros comandos <code>docker compose</code> falharam por não haver <code>daemon</code> em execução.	Inicialização do Docker Desktop e confirmação da subida com <code>docker compose ps</code> antes de prosseguir.
2	Conectividade SSH entre containers. O <code>mpirun</code> só consegue lançar processos remotos se o master autenticar nos workers sem senha, e por padrão o SSH ainda bloqueia aguardando confirmação interativa da <code>host key</code> .	Geração de par de chaves <code>ed25519</code> no <code>Dockerfile</code> , registro da pública em <code>authorized_keys</code> e configuração de <code>StrictHostKeyChecking no</code> com <code>UserKnownHostsFile=/dev/null</code> no <code>~/ .ssh/config</code> do <code>mpiuser</code> . Como todos os containers derivam da mesma imagem, compartilham o par de chaves.
3	Dependências repetidas por nó. O roteiro original exige <code>apt install</code> e <code>pip install</code> em cada um dos quatro containers, além de quatro <code>docker cp</code> por arquivo, o que é lento e propenso a divergência de versões entre nós.	Consolidação de todas as dependências e da cópia dos fontes no <code>Dockerfile</code> , reduzindo a implantação a um único <code>docker compose up -d --build</code> .
4	Particionamento de linhas com divisão não exata. A primeira versão perdia linhas quando N não era múltiplo de <code>size</code> .	Atribuição das linhas remanescentes ao último rank (<code>fim = N if rank == size - 1 else ...</code>), validada com o teste de $N = 4$ conferindo que a matriz remontada tem exatamente N linhas.
5	Medição de tempo inconsistente entre ranks. Sem sincronização, cada processo iniciava o cronômetro em um instante diferente, e o tempo reportado pelo rank 0 não refletia o custo real.	Inserção de <code>comm.Barrier()</code> imediatamente antes de <code>MPI.Wtime()</code> , alinhando o ponto de partida de todos os processos.
6	Volume excessivo de saída no terminal. A impressão integral da matriz C para $N = 1000$ gera um milhão de valores, dificultando a leitura do tempo e a captura de tela.	Rolagem do terminal até a linha final de tempo para o registro das evidências. Para execuções futuras, o mais adequado seria substituir a impressão da matriz por um <code>checksum</code> (<code>sum(sum(linha) for linha in C)</code>), que valida o resultado em uma única linha.
7	Sementes idênticas no Monte Carlo. Com a mesma semente em todos os ranks, os quatro processos gerariam sequências idênticas e a estimativa não melhoraria com mais processos.	Uso de <code>seed + rank</code> como semente de cada processo, mantendo reprodutibilidade e independência estatística.

7. Uso de Inteligência Artificial

Em atendimento à exigência de transparência do enunciado, declara-se o uso de ferramentas de IA generativa durante a realização desta atividade.

7.1 Ferramentas consultadas

- **Claude (Anthropic)** — esclarecimento de sintaxe do `mpi4py`, interpretação de mensagens de erro do Docker e do OpenMPI, e revisão da organização e da redação deste relatório.

7.2 Etapas em que a IA auxiliou

Etapa	Uso
Entendimento conceitual	Diferença entre as interfaces de alto nível (<code>bcast</code> , <code>gather</code> , <code>reduce</code>) e bufferizada (<code>Bcast</code> , <code>Gather</code> , <code>Reduce</code>) do <code>mpi4py</code> , e o papel do <code>pickle</code> no overhead de comunicação.
Depuração	Interpretação de erros de conexão SSH entre containers e da recusa do <code>mpirun</code> em lançar processos remotos.
Revisão de código	Verificação da lógica de particionamento de linhas e do posicionamento correto do <code>comm.Barrier()</code> em relação ao <code>MPI.Wtime()</code> .
Estruturação do relatório	Organização das seções conforme o roteiro do enunciado e revisão da clareza da análise de desempenho.

7.3 Prompts principais utilizados

- "Explique o motivo do erro X ao usar MPI neste código."
- "Qual a diferença entre `comm.bcast` e `comm.Bcast` no `mpi4py` e quando cada um deve ser usado?"
- "Por que o `mpirun` não consegue iniciar processos nos workers via SSH no Docker?"
- "Ajude-me a organizar melhor os resultados deste laboratório."

7.4 Análise crítica

O uso da IA foi restrito a apoio conceitual e de depuração. **A escrita do código, a execução dos programas no cluster, a coleta dos tempos e a captura das evidências foram realizadas manualmente no ambiente do laboratório**, e todas as sugestões recebidas foram conferidas contra o código real e contra a saída efetiva do terminal.

Quanto à precisão das respostas, observaram-se os seguintes pontos:

- **Respostas confiáveis** para conceitos consolidados do padrão MPI (semântica das operações coletivas, papel do rank, diferença entre as duas interfaces do `mpi4py`). Essas informações foram confirmadas na documentação oficial.
- **Imprecisões em detalhes de ambiente.** Sugestões relativas à configuração de SSH em containers e a flags do `mpirun` nem sempre correspondiam ao comportamento observado na imagem `ubuntu:22.04` com OpenMPI; foi necessário testar e ajustar manualmente até a configuração funcionar.
- **Tendência a superestimar o ganho da distribuição.** Explicações genéricas sobre MPI presumem *speedup* próximo do linear, o que não se verificou no Exercício 1. A contradição entre a expectativa teórica e a medição real só foi resolvida pela minha própria análise sobre o ambiente (containers no mesmo host, serialização `pickle`, laço interpretado). Este foi o aprendizado mais relevante da atividade:

a IA descreve o comportamento ideal do modelo, não o comportamento do sistema concreto — a interpretação dos números medidos é uma responsabilidade que não pode ser delegada.

- **A IA não identificou espontaneamente o viés metodológico** entre os *baselines* NumPy e a implementação MPI em Python puro. Essa discrepância só foi percebida ao confrontar a tabela com o crescimento esperado de um algoritmo $\mathcal{O}(N^3)$, notando que os tempos de $N = 600$ e $N = 1000$ eram menores que o de $N = 300$.

8. Conclusão

A atividade permitiu exercitar na prática os três padrões fundamentais de comunicação coletiva do MPI. No Exercício 1, **bcast** replicou as matrizes de entrada e **gather** reuniu as fatias de C calculadas independentemente por cada rank. No Exercício 2, **reduce** com **MPI.SUM** agregou os contadores locais em uma única operação.

O aprendizado central, contudo, não está na sintaxe, mas na **relação entre granularidade do trabalho e custo de comunicação**. Os dois exercícios funcionaram como contraexemplos recíprocos: o Monte Carlo, com comunicação constante e independente do tamanho do problema, beneficiou-se claramente da distribuição; a multiplicação de matrizes, que exige transmitir $\mathcal{O}(N^2)$ elementos serializados em ambas as direções, foi penalizada. MPI não é uma otimização automática — é uma ferramenta cujo ganho depende de o problema ter razão cálculo/comunicação favorável e de haver hardware físico adicional a ser agregado, condição que um cluster simulado em um único host não oferece.

Foram identificados, também, caminhos concretos de melhoria: substituir as operações de alto nível por suas contrapartes bufferizadas sobre **numpy.ndarray**, eliminando a serialização **pickle**; usar **scatter** para as linhas de A em vez de **bcast** da matriz inteira, reduzindo o tráfego; e uniformizar as implementações dos três modelos para que a comparação de desempenho meça efetivamente o efeito da paralelização.

9. Referências

1. MPI FORUM. **MPI: A Message-Passing Interface Standard, Version 4.1**. Knoxville: University of Tennessee, 2023. Disponível em: <https://www.mpi-forum.org/docs/>.
2. DALCÍN, L. et al. **MPI for Python (mpi4py) Documentation**. Disponível em: <https://mpi4py.readthedocs.io/>.
3. THE OPEN MPI PROJECT. **Open MPI Documentation**. Disponível em: <https://www.open-mpi.org/doc/>.
4. METROPOLIS, N.; ULAM, S. The Monte Carlo Method. **Journal of the American Statistical Association**, v. 44, n. 247, p. 335–341, 1949.
5. DOCKER INC. **Docker Compose Documentation**. Disponível em: <https://docs.docker.com/compose/>.
6. MATERIAL DIDÁTICO DA DISCIPLINA. **Atividade Lab — Computação Distribuída com MPI: Multiplicação de Matrizes e Método de Monte Carlo**. Prof. Alcides / Prof. Mário. FCI — Universidade Presbiteriana Mackenzie, 2026.